



How to train a MoE-LLM in 2026

Kirill Gelvan, Ai Agents, 19.03.2026

2 main pages

nanoMoE

Megatron MoE

Deep (Learning) Focus

nanoMoE: Mixture-of-Experts (MoE) LLMs from Scratch in PyTorch

An introductory, simple, and functional implementation of MoE LLM pretraining...



CAMERON R. WOLFE, PH.D.
MAR 10, 2025



177



13



29

Share

<https://cameronrwolfe.substack.com/p/nano-moe>

Quick Start Guide

Basic MoE Training in Megatron-LM

To train a top-2 MoE model with 8 experts and auxiliary loss, add the following arguments to your megatron training script:

```
## Set MoE Hidden size
--num-experts 8
--moe-shared-expert-intermediate-size: 2048
## Set router config
--moe-router-load-balancing-type aux_loss
--moe-router-topk 2
--moe-aux-loss-coeff 1e-2
## Set token dispatcher
--moe-token-dispatcher-type alltoall
```

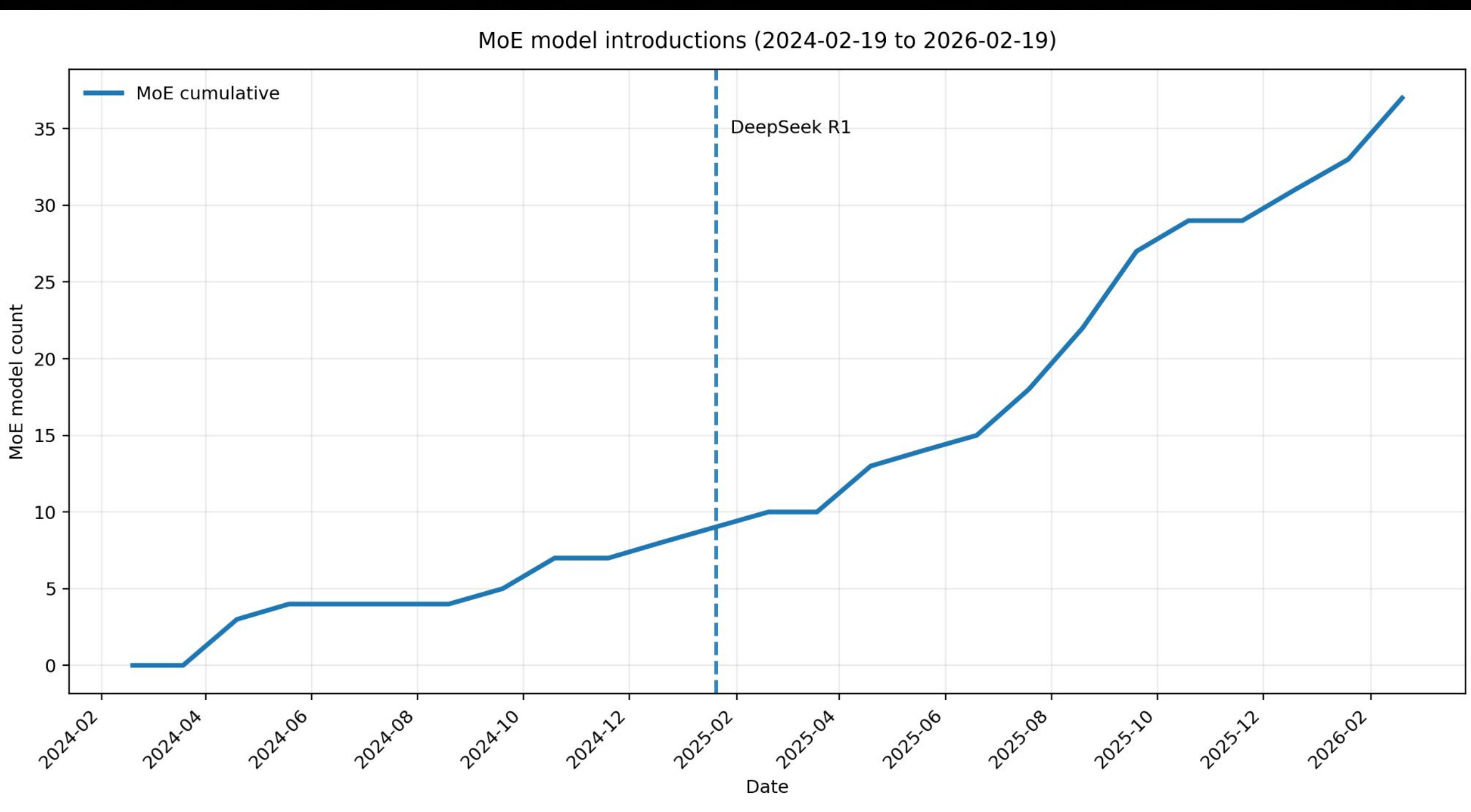
Detailed documentation for each feature is available in the [Feature Documentation](#) section.

<https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/features/moe.html>

Part 1.

**What is MoE & general
params**

Why MoE now



- 37+ models in 2 years
- Sharp spike after DeepSeek R1
- Mixtral → DeepSeek-V2/V3 → Qwen → DBRX

Models 2,813

Active filters: moe

If you're training LLMs, you will hit MoE.

MoE in 60 seconds

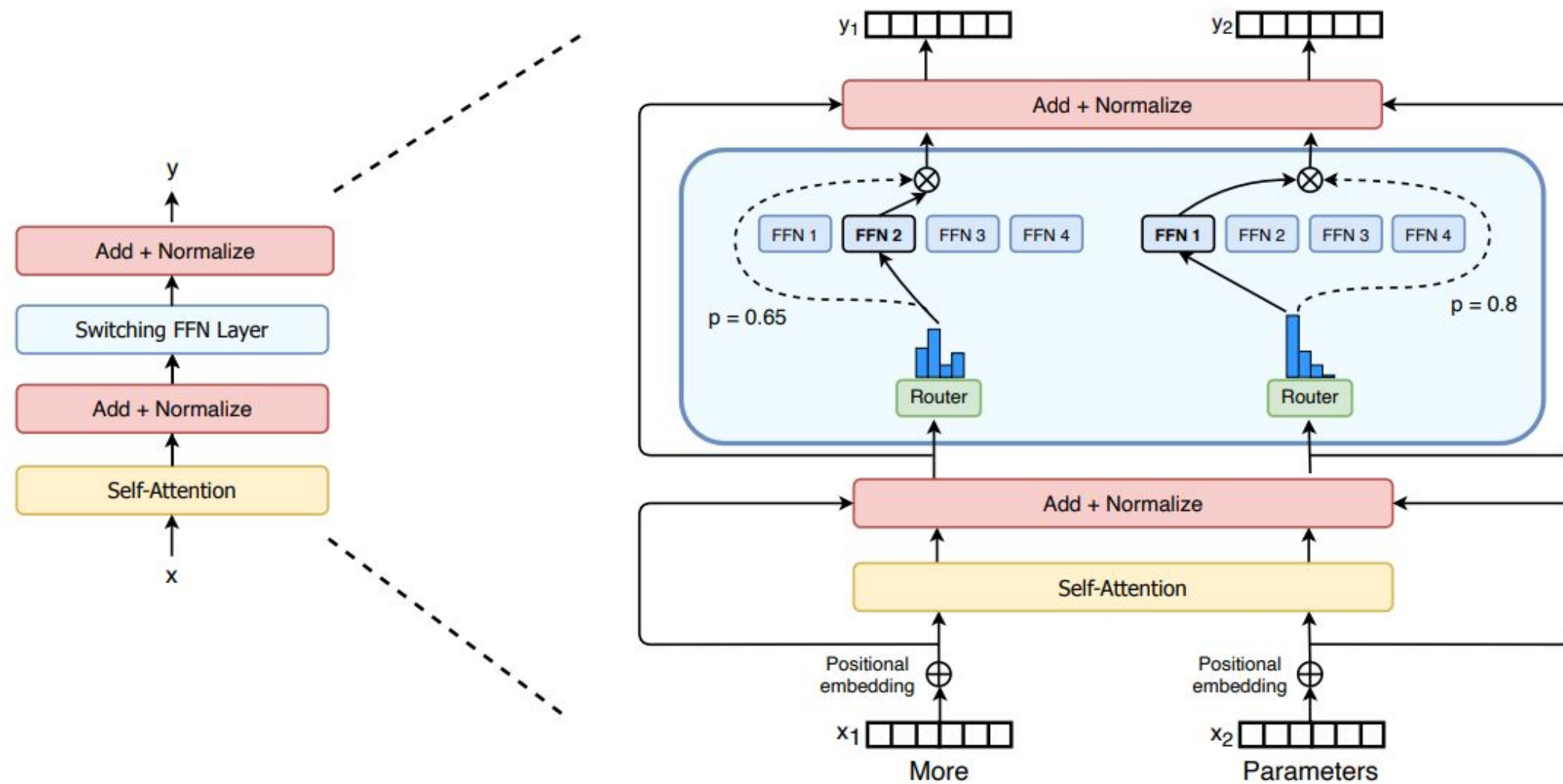


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

- Replace dense FFN $\rightarrow$ N expert FFNs + router
- Router picks **top-K** per token
- Output = weighted sum of chosen experts

You know the theory. Let's talk practice.

MoE: wins and traps

Why it's attractive

- Higher capacity, similar active compute
- Same quality, 2× faster training
- Only K of N experts fire at inference

What breaks

- ✗ Memory is NOT sparse (all optimizer states)
- ✗ Routing adds instability + new losses
- ✗ Communication becomes token-dependent
- ✗ Fine-tuning is more fragile

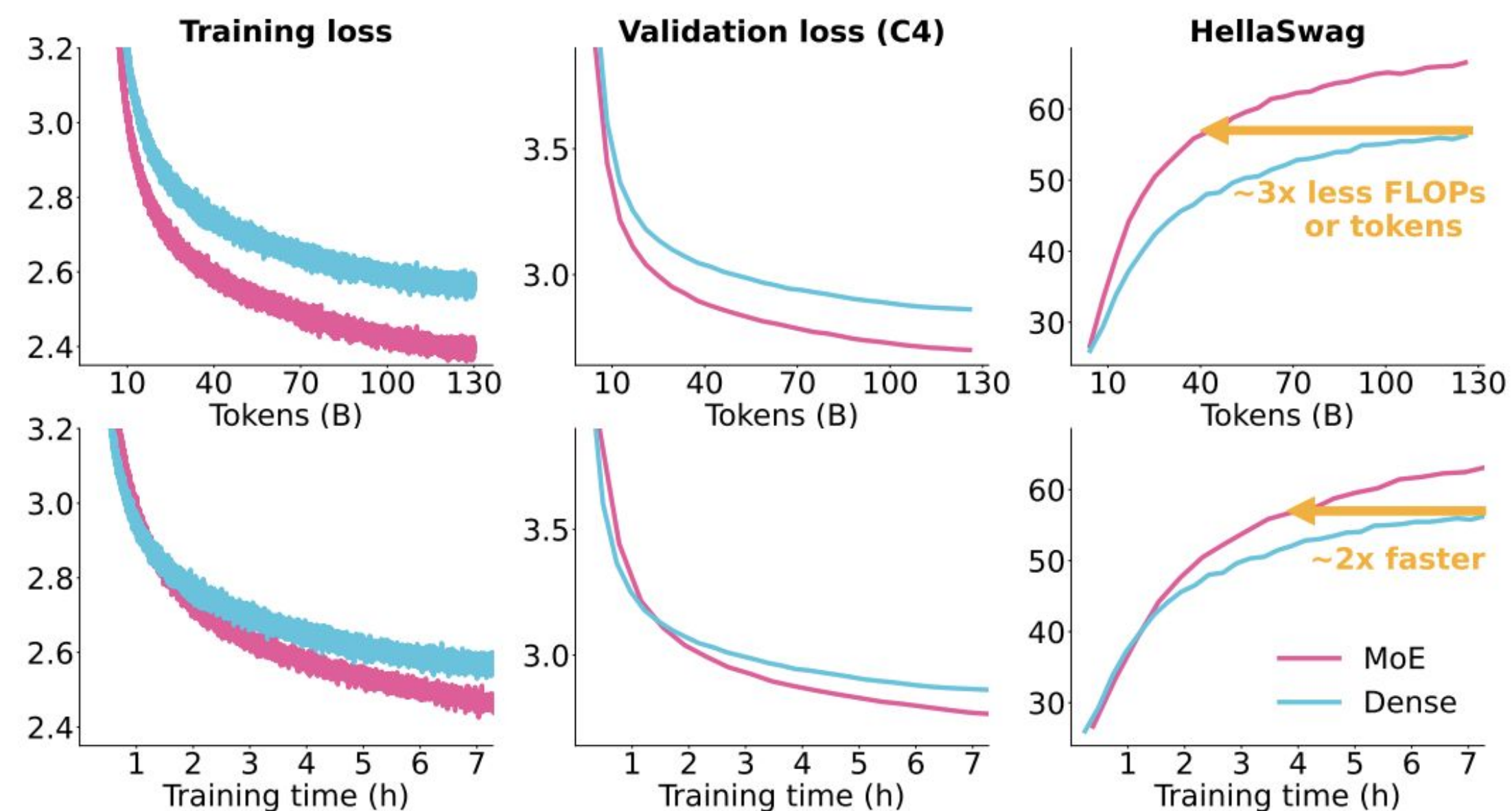


Figure 4: **MoE vs. Dense.** We train a 1.3B parameter dense model and a 1.3B active, 6.9B total parameter MoE model, each on 128 H100 GPUs. Apart from MoE-related changes, we train both

Routing: how tokens find experts

router logits



top-K selection



mask → softmax



sparse routing weights

Problem Statement

- 4 Tokens
- 4 Experts
- 2 Slots per Expert

Which *token* routes to which *expert* and in what *slot*?

TOKENS

	d0	d1	d2
T0	-0.5	0.3	0.5
T1	-0.3	-0.8	-0.1
T2	-0.4	0.3	-0.8
T3	-0.5	-0.5	0.2

EXPERTS

	S0	S1
E0		
E1		
E2		
E3		

Normalize logits

TOP-K ROUTER LOGITS SCATTERED with -INF

	E0	E1	E2	E3
T0	-inf	0.9	-inf	0.6
T1	0.8	-inf	0.5	-inf
T2	-inf	0.5	-inf	0.8
T3	-inf	0.7	-inf	0.6

SOFTMAX

NORMALIZED TOKEN WEIGHTS

	E0	E1	E2	E3
T0	0.0	0.5	0.0	0.4
T1	0.5	0.0	0.4	0.0
T2	0.0	0.4	0.0	0.5
T3	0.0	0.5	0.0	0.4

Small score changes → routing changes.
Stability matters.

```
--moe-router-topk 2 --moe-router-score-function softmax
```


Capacity factor & token dropping

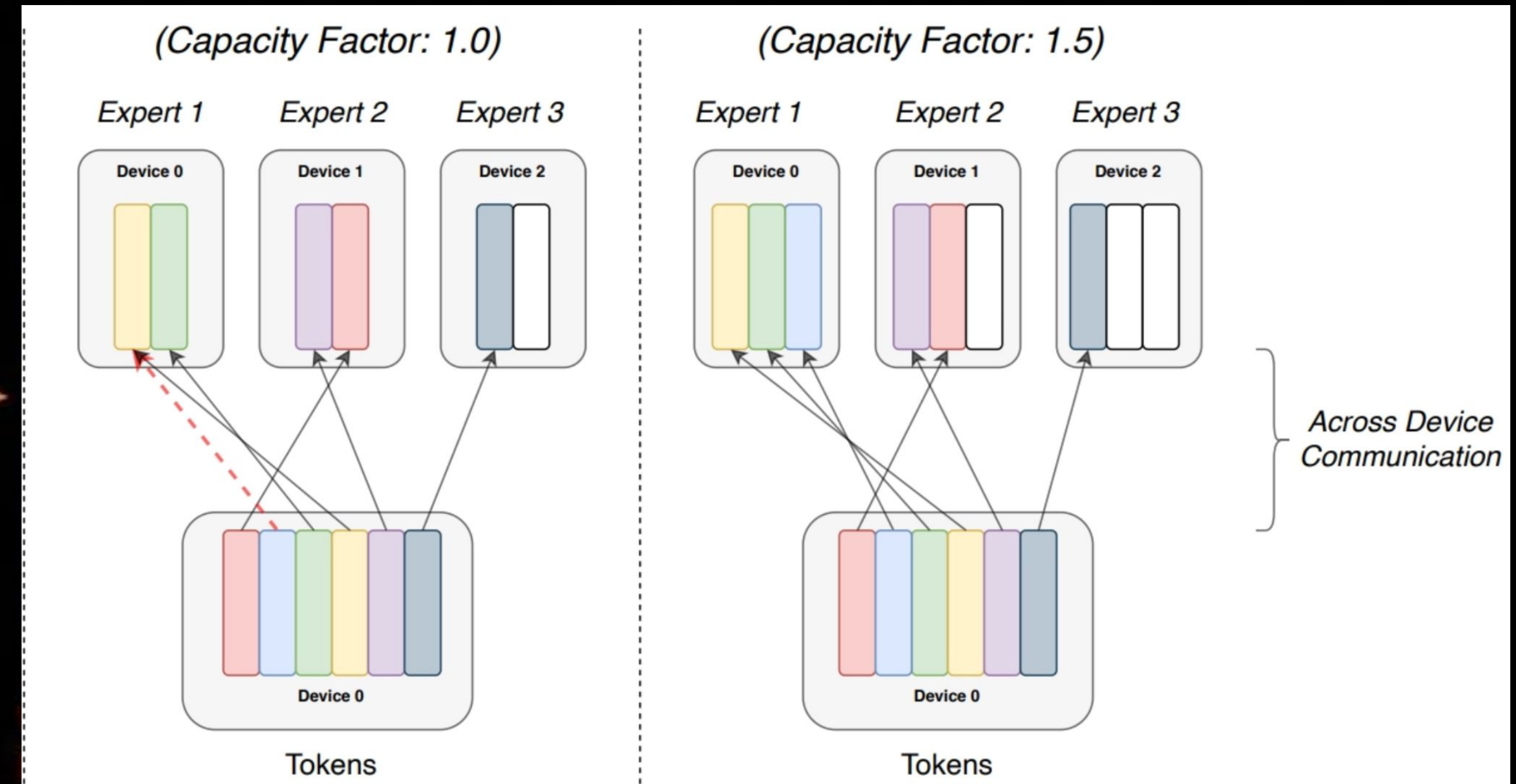
$$\text{Expert Capacity} = \frac{K \cdot B \cdot C}{N} \cdot \text{Capacity Factor}$$

Number of Active Experts (points to K)

Total Number of Tokens in Batch (points to B)

Total Number of Experts (points to N)

- CF=1.0 assumes perfect balance → tokens dropped
- CF>1 = buffer for imbalance, costs memory
- CF ≈ 1.25 is a strong default (ST paper)



Core MoE Functionality

- ☒ Token dropless MoE - Advanced routing without token dropping
- ☒ Top-K Router with flexible K selection
- ☒ Load balancing losses for expert load balancing optimization

```
--moe-expert-capacity-factor 1.25  
--moe-token-drop-policy probs
```


Load balancing + router stability

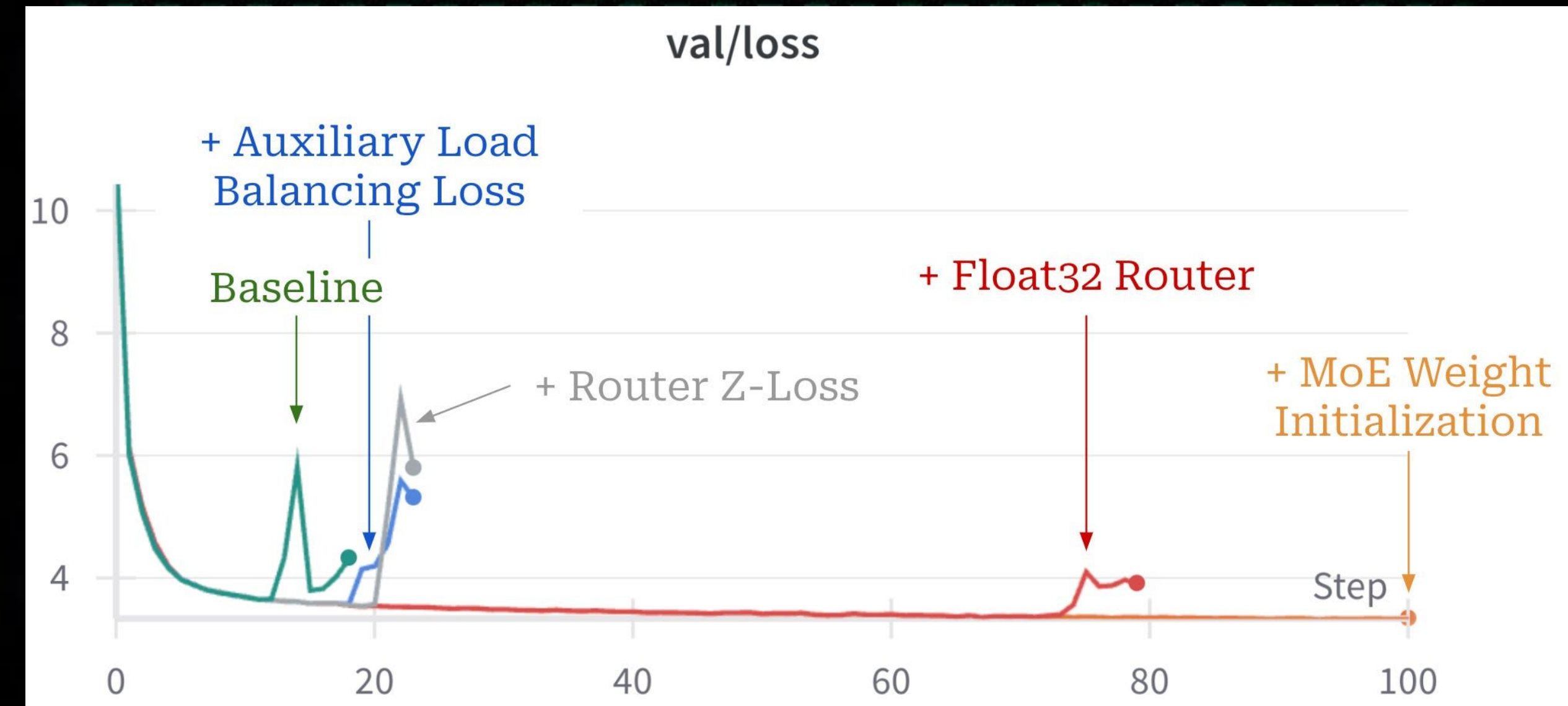
Without intervention, router collapses to 1-2 experts.

aux_loss — penalize imbalanced assignment

z-loss — penalize large router logits

FP32 router — bf16 kills accuracy at scale

jitter / noisy gating — input noise



$$L_{\text{full}}(\mathcal{X}) = \underbrace{L_{\text{LM}}(\mathcal{X})}_{\text{Standard Language Modeling Loss}} + \underbrace{w_{\text{load-balance}} \cdot L_{\text{load-balance}}(\mathcal{X})}_{\text{Weighted load balancing loss}} + \underbrace{w_{\text{router-z}} \cdot L_{\text{router-z}}(\mathcal{X})}_{\text{Weighted router-z loss}}$$

```
--moe-aux-loss-coeff 1e-2 --moe-z-loss-coeff --moe-router-dtype fp32 --moe-input-jitter-eps
```

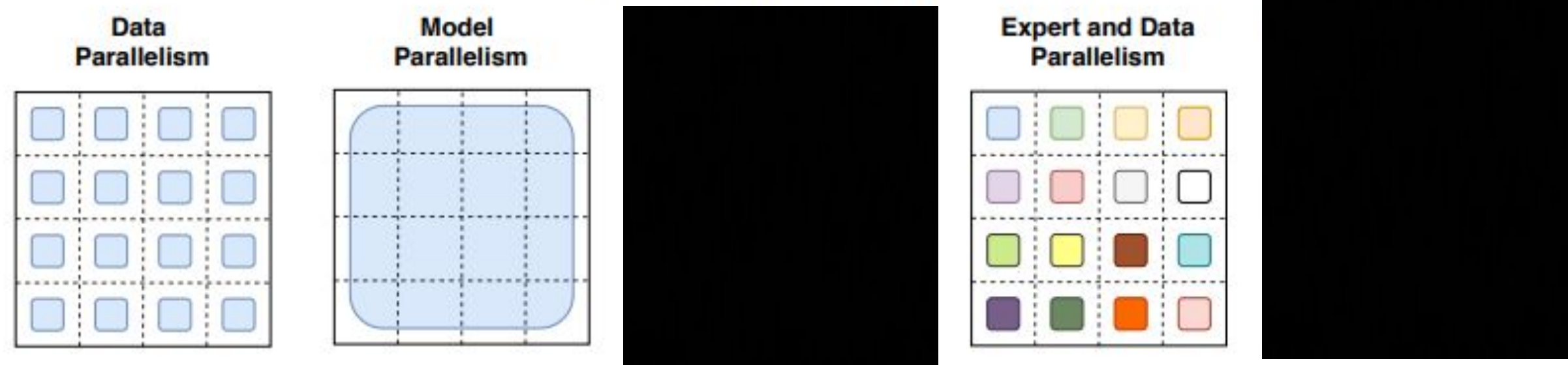

Part 2.

How to run this on GPUs

& why is this hard

Parallelism mental model

How the **model weights** are split over cores



How the **data** is split over cores



DP — replicate model, split data

TP — split one layer's weights across GPUs

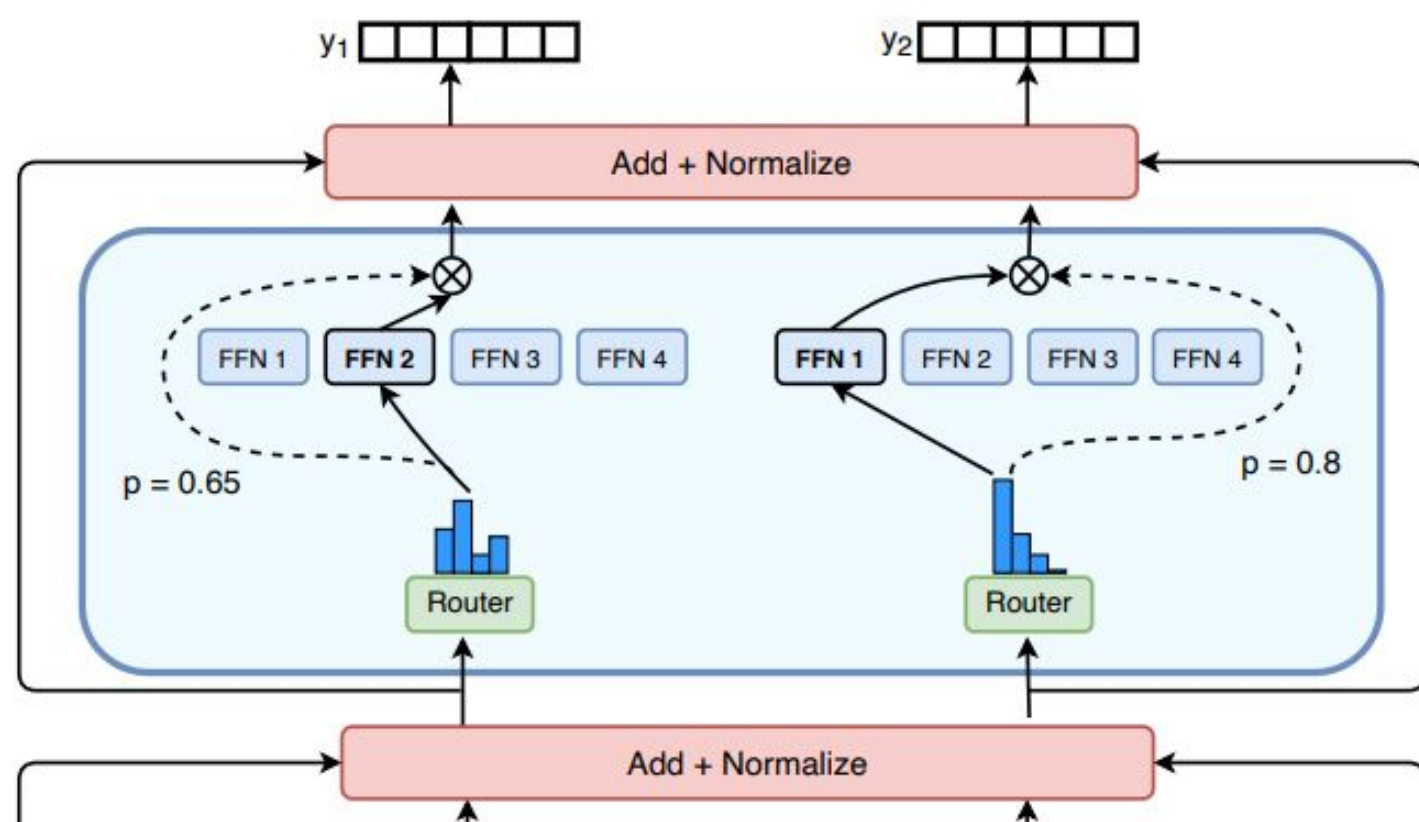
PP — split layers across GPUs

EP — split experts across GPUs (MoE-specific)

CP — split long sequences ($\geq 8K$)

Practical rules

1. Maximize DP, minimize TP/EP/PP
2. $EP \times TP \leq 8$ (stay within NVLink node)
3. PP for multi-node scaling
4. Prefer EP over TP for expert layers
5. CP mainly for sequences 8K+



Token dispatch

Tokens physically move to the expert's GPU → compute → move back.

Dispatcher	Description	Best For	Config
alltoall	NCCL-based All-to-All communication for token exchange	Standard EP > 1 setups	<code>--moe-token-dispatcher-type alltoall</code>
FlexDispatcher with DeepEP backend	Removes redundant tokens during cross-node communication, fuses intra/inter-node communication into single kernel	Cross-node EP, fine-grained MoE (DeepSeek-V3)	<code>--moe-token-dispatcher-type flex</code> <code>--moe-flex-dispatcher-backend deepEP</code>
FlexDispatcher with HybridEP backend	NVIDIA's optimized dispatcher using TMA and IBGDA, fewer SMs, native MNNVL support	GB200 NVL72, Multi-Node NVLink	<code>--moe-token-dispatcher-type flex</code> <code>--moe-flex-dispatcher-backend hybridEP</code>
allgather	Gathers all tokens to each GPU, no inter-GPU token movement	TP-only setups, small EP, large Top-K	<code>--moe-token-dispatcher-type allgather</code>

alltoall

NCCL-based, your default for EP > 1

allgather

gathers all tokens to every GPU, for small EP / TP-only

flex backends

DeepEP, HybridEP — cross-node / GB200

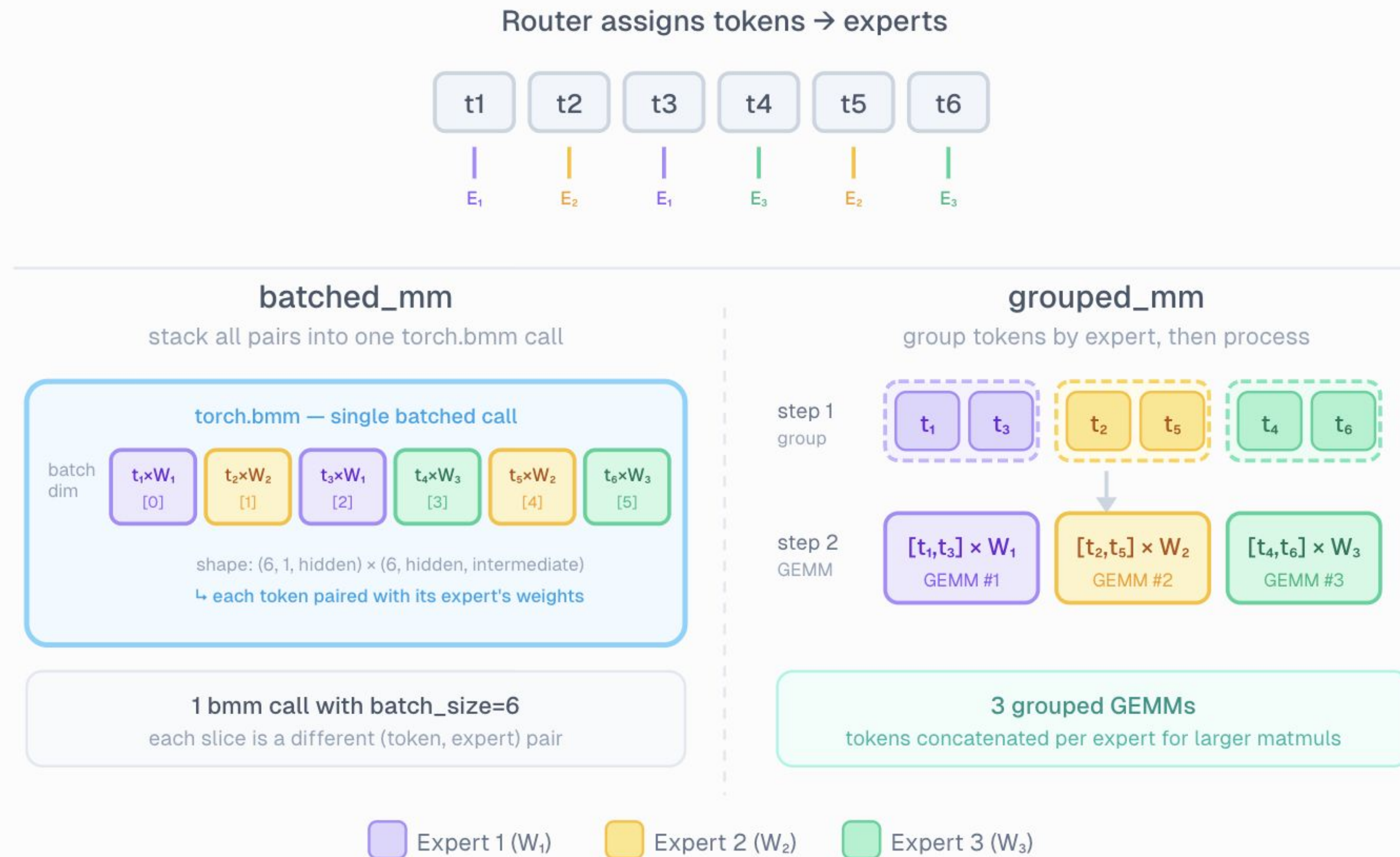
```
--moe-token-dispatcher-type alltoall
```


Grouped GEMM & fused kernels

Naive MoE = for-loop over experts = terrible GPU util.

```
for expert_idx in range(num_experts):  
    tokens_for_expert = tokens[expert_idx]  
    output[expert_idx] = tokens_for_expert @ weights[expert_idx]
```

```
# maybe 50 tokens  
tokens[expert_idx]
```



batched_mm stacks all token×expert pairs into one bmm call.
grouped_mm concatenates tokens per expert for fewer, larger GEMMs.

Memory + overlap + profiling

Run profiler



Identify bottleneck



Apply targeted fix

Quick Start Guide

Basic MoE Training in Megatron

To train a top-2 MoE model with 8 experts and megatron training script:

```
## Set MoE Hidden size
--num-experts 8
--moe-shared-expert-intermediate-size: 2
## Set router config
--moe-router-load-balancing-type aux_loss
--moe-router-topk 2
--moe-aux-loss-coeff 1e-2
## Set token dispatcher
--moe-token-dispatcher-type alltoall
```

Detailed documentation for each feature is available

Step 3: Enable Performance Features Based on Profiling Bottlenecks

Memory Bottleneck

Communication

Bottleneck

CPU Overhead Bottleneck

Computation Bottleneck

Please use profiler 🙏

LoRA + MoE: what's tricky

Option A: fused adapter

One LoRA pair on the whole
[num_experts, hidden, ffn] tensor

Option B: per-expert adapters

Each expert gets its own A_i, B_i
128 experts $\times$ 48 layers = thousands
of tiny matrices

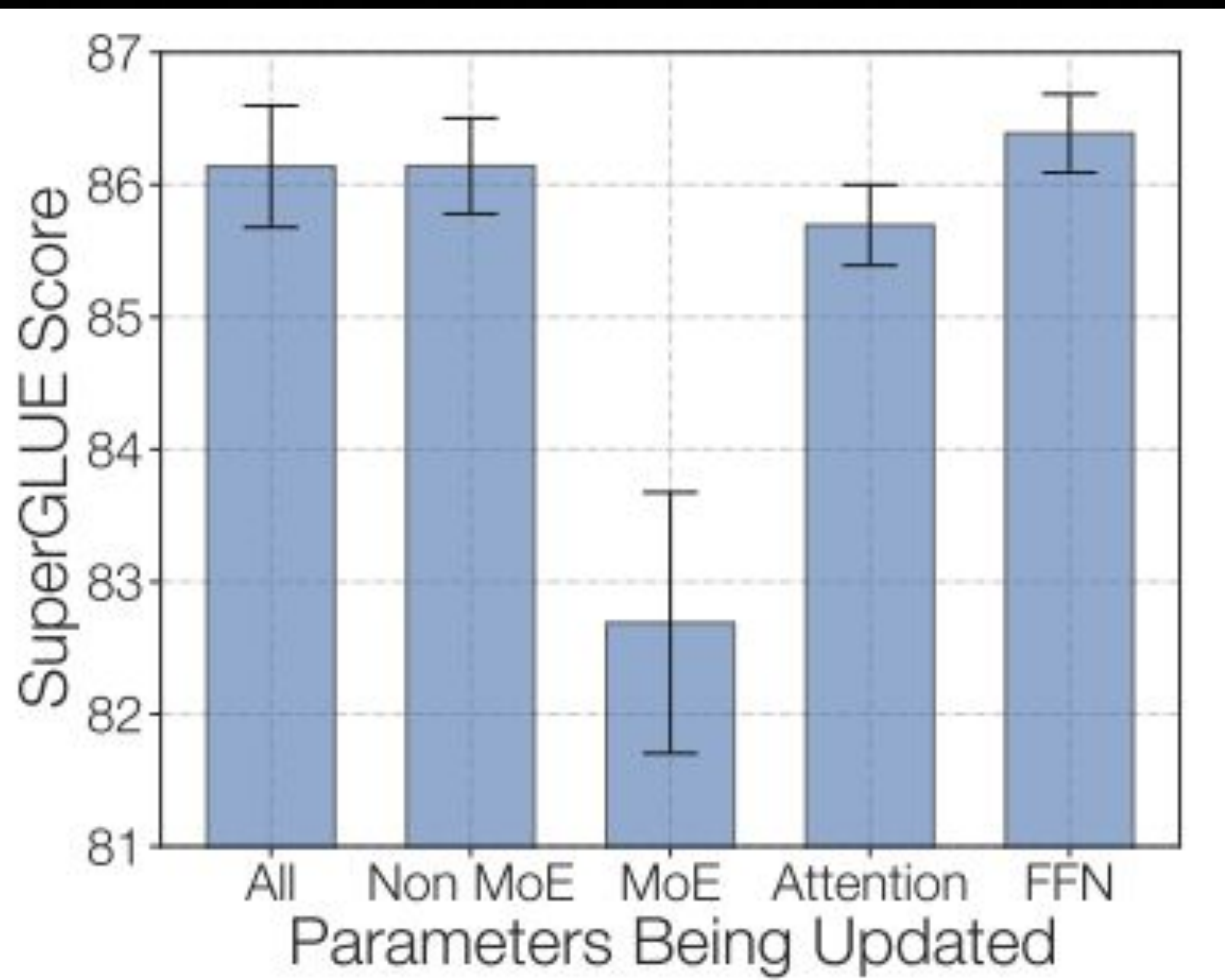
The EP complication

- Breaks grouped-GEMM
- Gradients are harder to compute and communicate
- Only possible in Megatron for now

Part 3.

How to Fine-Tune MoE-LLM

Fine-tuning MoE: what's different



All params	~86
Non-MoE only	~86
FFN only	~86.5 ★
Attention only	~85.7
MoE-only	~82.5

- MoE-only FT is significantly worse
- Overfitting is the main enemy, not capability

High-confidence defaults: your first run

- ✓ Keep pretrained top-K routing
- ✓ Router in FP32 always
- ✓ aux_loss (coeff $\sim 1e-2$)
- ✓ z-loss for stability
- ✓ AllToAll dispatcher ($EP > 1$)
- ✓ Grouped GEMM
- ✓ Per-layer load monitoring
- ✓ CF ≈ 1.25 (or token-droptless)

```
--moe-router-dtype fp32 --moe-aux-loss-coeff 1e-2 --moe-grouped-gemm --moe-per-layer-logging
```


Experiment branches: what to try next

Promising but not settled defaults.

- Freeze router (or separate smaller LR in fp32) — with LoRA it's frozen by default
- ESFT: identify active experts for your task → train only those
- aux_loss off during FT (try both on/off)
- Extra dropout (careful: too much hurts)
- LoRA vs full FT (skip 4-bit QLoRA for MoE)
- adamw_8bit optimizer

Key takeaways & my experiments

- MoE = compute-efficient but infrastructure-complex
- Router stability is everything: fp32, aux_loss, z_loss, jitter
- Don't tune only MoE weights — train attention/FFN parts
- Router handling (freeze / low LR / ESFT) = experiment branches
- EP > TP for expert layers, keep EP × TP within one node
- Grouped GEMM is mandatory for performance
- Memory is the bottleneck — distributed opt, precision-aware, recomputation
- Sweep LR and batch size — sparse ≠ dense
- Profile first, optimize second
 - + LoRA is complicated, but works in Megatron!